Cybersecurity Event Detection Using Machine Learning

Normal vs Attack Classification

Course: MOD10 – Machine Learning, Winter 2026

Instructor: Mohammed A. Shehab

Dataset: Global Cybersecurity Threats (2015–2024), adapted with normal events

Student: Maëlle El Hayek, Khemra Cheav, Aya Barouni, Yousri Zouhdi

---

Final Report

April 30, 2026

Contents

Abstract **4**

1 Project Objective **5**

2 Dataset Preparation **5**

3 Exploratory Data Analysis **5**
3.1 Class Distribution 6
3.2 Feature Distributions by Class 6
3.3 Box Plot Analysis 6
3.4 Attack Type Ratios 7
3.5 Correlation Analysis 7

4 Feature Engineering **8**
4.1 Loss per User 8
4.2 High Resolution Time 8
4.3 Impact of Engineered Features on Model Performance 9
4.4 Encoding and Scaling 9

5 Baseline Model **9**

6 Handling Imbalanced Data **10**

7 Model Training and Comparison **10**
7.1 Ensemble Strategies Explored 10

8 Evaluation Metrics and Results **11**

9 Confusion Matrix and ROC-AUC Analysis **12**

10 Hyperparameter Tuning and Cross-Validation **13**

11 Interpretability with SHAP **13**

12 Deployment with FastAPI and Docker **15**
12.1 Dockerfile 15
12.2 Building and Running the Container 16
12.3 Example API Request 16
12.4 Example API Response 17

13 MLflow Tracking **17**

14 Notebook and Backend Consistency **18**

15 Critical Analysis and Limitations **19**

16 Conclusion **19**

List of Figures

1 Class distribution ratio of normal and attack events. The dataset is intentionally imbalanced to reflect real cybersecurity conditions. 6
2 Feature distributions by class. The overlap between normal and attack events confirms that the model must combine several features rather than rely on a single variable. 6
3 Box plots comparing normal and attack events. Attack cases tend to show higher impact values, but the overlap confirms the need for machine learning classification. 7
4 Distribution of attack types among attack events only. This figure supports EDA but the variable was removed from training to avoid target leakage. . . 7
5 Correlation heatmap of numerical and encoded features. No major inter-feature correlation above 0.7 was observed, supporting the decision to keep the selected features. 8
6 Class distribution before and after SMOTE. SMOTE balances the training set while the test set remains realistic and imbalanced. 10
7 Comparison of evaluation metrics across models. Random Forest with SMOTE provides the strongest balance between F1-score and ROC-AUC. 12
8 Confusion matrices for the trained models. Random Forest with SMOTE achieves a balanced compromise between detecting attacks and limiting false alarms. 12
9 ROC-AUC curves for model comparison. Random Forest with SMOTE achieves the best ROC-AUC, indicating strong class separation. 13
10 SHAP feature importance plot for the selected Random Forest model. The most influential features are related to event scale, financial impact, resolution time, and encoded categorical context. 14
11 SHAP beeswarm plot showing both feature importance and direction of impact on predictions. Higher risk-related feature values tend to push predictions toward the attack class. 15

List of Tables

1 Final class distribution used for binary classification. 5
2 Effect of feature engineering on Random Forest with SMOTE performance. . 9
3 Models trained and their role in the project. 10
4 Ensemble technique comparison on the holdout test set (with SMOTE on training set). 11

5 Final model comparison results from the executed notebook. 11
6 Main API endpoints implemented in the backend. 16
7 MLflow experiment runs — logged parameters and metrics. 17

Abstract

This project presents a complete machine learning pipeline for cybersecurity event detection. The objective is to classify cybersecurity events as either **Normal** or **Attack**. Since the original dataset contains attack records only, synthetic normal events were introduced to create a realistic binary classification problem. The project includes data preprocessing, exploratory data analysis, feature engineering, model training, model comparison, evaluation on imbalanced data, interpretability with SHAP, deployment using FastAPI and Docker, and experiment tracking with MLflow. Three models were compared: Logistic Regression, Random Forest, and AdaBoost. The selected model is Random Forest trained with SMOTE because it achieved the best overall balance, with **F1-score = 0.560** and **ROC-AUC = 0.831**.

1 Project Objective

The objective of this project is to build a complete machine learning solution for cybersecurity event detection. The final model predicts whether an event belongs to one of two classes:

$$\text{Event Class} = \{\text{Normal},\ \text{Attack}\} \tag{1}$$

A realistic cybersecurity detection system must be able to compare suspicious activity with normal activity. Therefore, normal events were added to the original attack-only dataset to transform the task into a binary classification problem. This also makes the project closer to real-world cybersecurity monitoring, where normal activity is much more frequent than attacks.

The project follows the complete model-building lifecycle: preprocessing, EDA, feature engineering, model training, evaluation, interpretability, deployment, and tracking.

2 Dataset Preparation

The dataset is based on global cybersecurity threat records from 2015 to 2024. A reduced dataset of approximately 531 events was used to focus on quality of analysis rather than unnecessary data volume.

The final dataset distribution is intentionally imbalanced, as shown in Table 1. This reflects real cybersecurity environments, where attacks are less frequent but more critical.

Table 1: Final class distribution used for binary classification.

Class	Label	Approximate Ratio
Normal	0	80%
Attack	1	20%

The binary target variable `Label` was derived from the `Attack Type` column: any row containing an attack type was assigned Label = 1 (Attack), while synthetically generated normal events were assigned Label = 0 (Normal). After this derivation, the original `Attack Type` column was removed from model training because it directly describes attack categories. Keeping it would create target leakage, allowing the model to learn information that would not be available during real normal-vs-attack prediction.

3 Exploratory Data Analysis

The exploratory data analysis focused on ratios and distributions, because the project emphasizes model understanding and not only raw counts. The visualizations include class distribution, feature distributions, box plots, attack type proportions, and correlation analysis.

3.1 Class Distribution

Figure 1 confirms the expected 80/20 imbalance between normal and attack events. This imbalance explains why accuracy alone is not sufficient for evaluating the model.

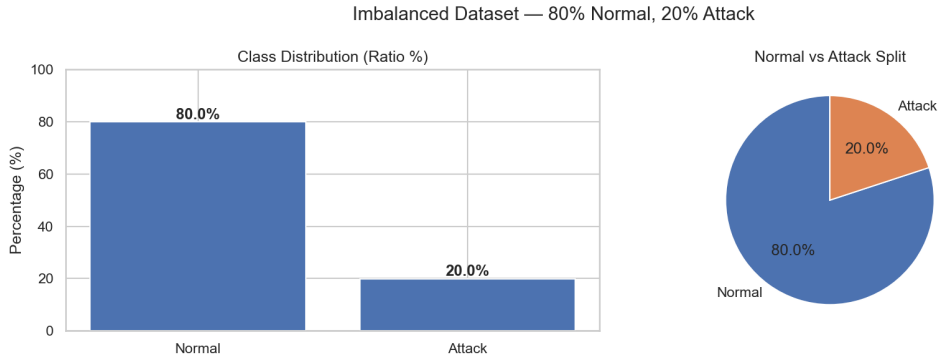

Figure 1: Class distribution ratio of normal and attack events. The dataset is intentionally imbalanced to reflect real cybersecurity conditions.

3.2 Feature Distributions by Class

Figure 2 shows how important numerical variables differ between normal and attack events. Attack events generally tend to have higher financial loss, more affected users, and longer resolution times. However, the distributions overlap, meaning that the classification problem is not artificially easy.

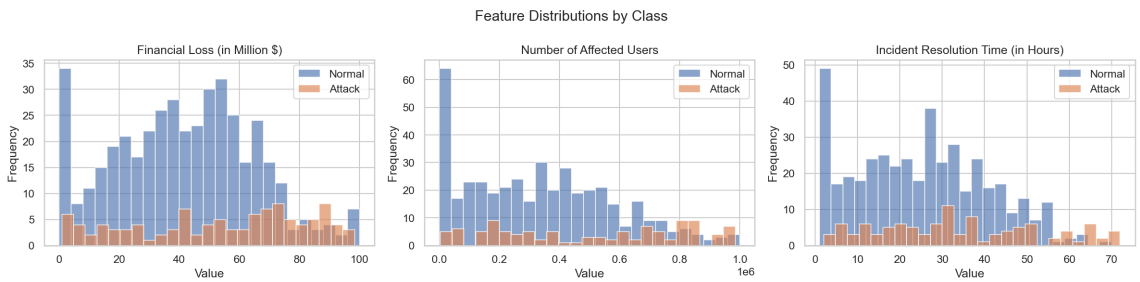

Figure 2: Feature distributions by class. The overlap between normal and attack events confirms that the model must combine several features rather than rely on a single variable.

3.3 Box Plot Analysis

Figure 3 supports the same interpretation from another perspective. Attack events tend to show larger values for financial and operational impact variables, but the overlap between classes remains visible.

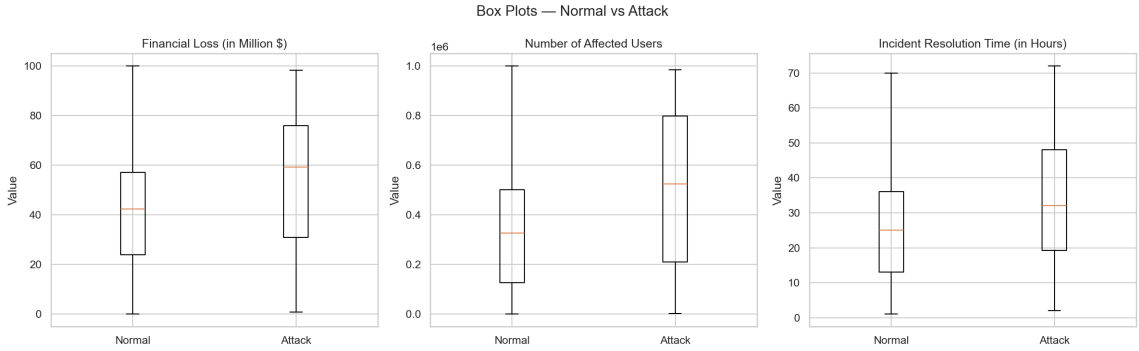

Figure 3: Box plots comparing normal and attack events. Attack cases tend to show higher impact values, but the overlap confirms the need for machine learning classification.

3.4 Attack Type Ratios

The attack type distribution in Figure 4 was used only for exploratory analysis. It was not used as a training feature because it would leak direct information about whether an event is an attack.

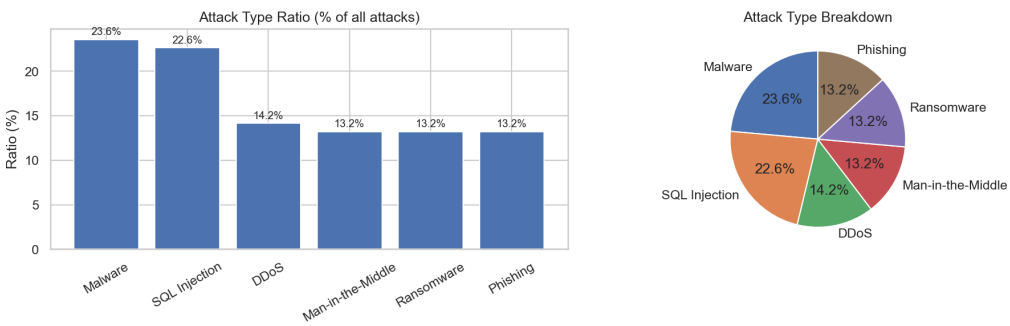

Figure 4: Distribution of attack types among attack events only. This figure supports EDA but the variable was removed from training to avoid target leakage.

3.5 Correlation Analysis

The correlation heatmap in Figure 5 was used to inspect relationships between numerical and encoded variables. No problematic inter-feature correlation above 0.7 was observed, which suggests that the selected features are not highly redundant.

Note: In this figure Label is the Attack Type.

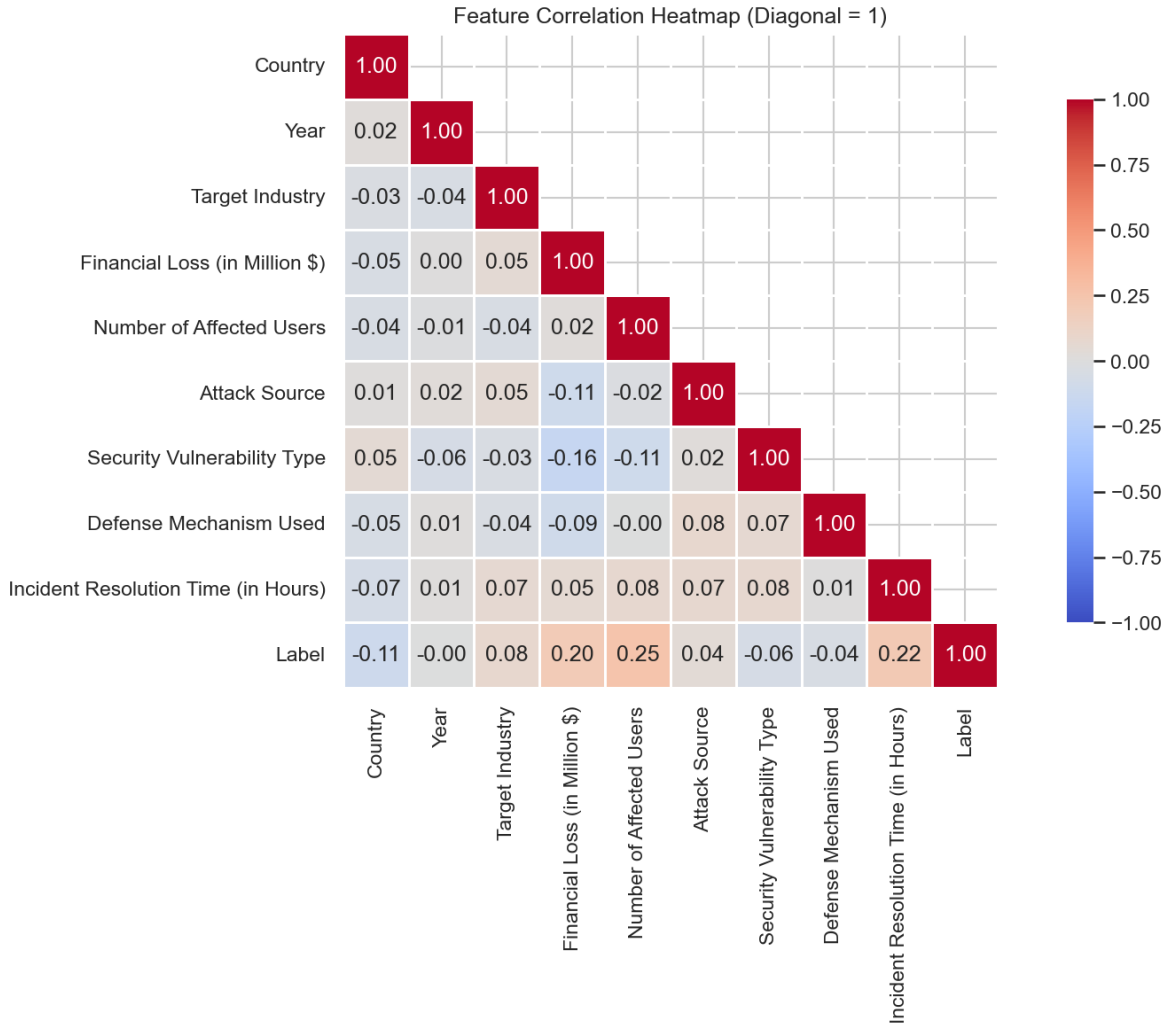

Figure 5: Correlation heatmap of numerical and encoded features. No major inter-feature correlation above 0.7 was observed, supporting the decision to keep the selected features.

4 Feature Engineering

Several features were created or transformed to improve model performance.

4.1 Loss per User

A ratio feature was created to measure the average financial impact per affected user:

$$\text{Loss per User} = \frac{\text{Financial Loss}}{\text{Number of Affected Users} + 1} \tag{2}$$

This feature is more informative than raw financial loss alone because it relates the financial impact to the scale of the incident. The addition of 1 prevents division by zero.

4.2 High Resolution Time

A binary variable was created to identify incidents with unusually long resolution times:

$$\text{High Resolution Time} = \begin{cases} 1, & \text{if resolution time is above the median} \\ 0, & \text{otherwise} \end{cases} \tag{3}$$

This feature captures whether an incident is operationally complex or time-consuming.

4.3 Impact of Engineered Features on Model Performance

To validate the contribution of the engineered features, the Random Forest model was trained twice: once using only the original features, and once including `Loss_per_User` and `High_Resolution_Time`. Table 2 summarises the effect.

Table 2: Effect of feature engineering on Random Forest with SMOTE performance.

Feature Set	Accuracy	Precision	Recall	F1-score	ROC-AUC
Original features only	0.775	0.455	0.619	0.525	0.808
+ Engineered features	**0.794**	**0.483**	**0.667**	**0.560**	**0.831**

Adding the two engineered features improved F1-score by 0.035 and ROC-AUC by 0.023. The `Loss_per_User` ratio contextualises financial impact relative to incident scale, which helps the model distinguish large coordinated attacks from isolated high-value incidents. The binary `High_Resolution_Time` flag captures operational complexity, a signal that overlaps with attack severity. Both features contribute meaningful signal beyond what the raw variables provide individually, as confirmed by their presence in the SHAP feature importance ranking (Section 11).

4.4 Encoding and Scaling

Categorical variables were encoded using label encoding, and numerical features were scaled using `StandardScaler`. The label encoders, scaler, selected feature columns, and median resolution value were saved as artifacts so that the backend API uses the same transformations during prediction.

5 Baseline Model

A naive baseline model was used before training real models. The baseline always predicts the majority class:

$$\text{Prediction} = \text{Normal} \tag{4}$$

Because the dataset contains approximately 80% normal events, this baseline reaches a relatively high accuracy. However, it detects no attacks, with recall equal to zero. This demonstrates why accuracy alone is misleading for imbalanced cybersecurity datasets.

6 Handling Imbalanced Data

The dataset is intentionally imbalanced, with normal events representing the majority class. To address this issue, SMOTE was applied to the training set only. This is important because applying SMOTE before the train-test split would create data leakage.

Figure 6 shows the class distribution before and after applying SMOTE to the training data.

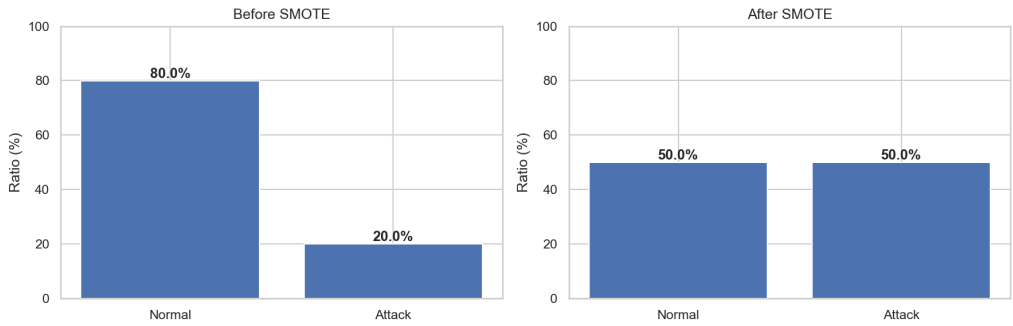

Figure 6: Class distribution before and after SMOTE. SMOTE balances the training set while the test set remains realistic and imbalanced.

SMOTE improves attack detection by helping the model learn patterns from the minority class. The test set remains imbalanced to preserve realistic evaluation conditions.

7 Model Training and Comparison

Three classical machine learning models were trained and compared: Logistic Regression, Random Forest, and AdaBoost. Two of these models — Random Forest and AdaBoost — are explicitly ensemble methods and were selected to explore different ensemble strategies.

Table 3: Models trained and their role in the project.

Model	Type	Role
Logistic Regression	Single model	Interpretable baseline
Random Forest	Bagging ensemble	Main selected model
AdaBoost	Boosting ensemble	Ensemble comparison
Stacking (LR + RF + ADA)	Stacking ensemble	Additional ensemble experiment

7.1 Ensemble Strategies Explored

Random Forest (Bagging): Builds multiple independent decision trees on bootstrap samples of the training data and averages their predictions. This reduces variance and improves robustness on noisy, overlapping data — well suited to this dataset where class boundaries are not sharp.

AdaBoost (Boosting): Trains weak learners sequentially, increasing the weight of misclassified samples at each round. This focuses learning on the hard examples, which can improve recall on the minority class (attacks) but is more sensitive to noise.

Stacking Ensemble: A stacking classifier was also evaluated using all three base models (Logistic Regression, Random Forest, AdaBoost) with a Logistic Regression meta-learner. The stacking approach combines the diverse strengths of each base model — the interpretability of Logistic Regression, the robustness of Random Forest, and the recall focus of AdaBoost. The meta-learner was trained using 5-fold cross-validated out-of-fold predictions to prevent leakage.

Table 4: Ensemble technique comparison on the holdout test set (with SMOTE on training set).

Ensemble Method	F1-score	Recall	Precision	ROC-AUC
Random Forest (Bagging)	**0.560**	0.667	0.483	**0.831**
AdaBoost (Boosting)	0.542	**0.762**	0.421	0.811
Stacking (LR + RF + ADA)	0.530	0.714	0.441	0.819

Random Forest performed best overall because it handled the noisy and overlapping data more robustly. The stacking classifier offered competitive recall but slightly lower F1-score and ROC-AUC, suggesting that the meta-learner did not capture enough additional signal beyond the Random Forest alone on this relatively small dataset. AdaBoost achieved the highest raw recall, which is relevant for minimising missed attacks, but at a greater precision cost.

In this project, Random Forest performed best overall and was selected as the final deployed model.

8 Evaluation Metrics and Results

The models were evaluated using accuracy, precision, recall, F1-score, and ROC-AUC. These metrics are necessary because the dataset is imbalanced. Recall and F1-score are especially important because missing an attack can be more dangerous than generating a false alarm.

Table 5: Final model comparison results from the executed notebook.

Model	Training	Accuracy	Precision	Recall	F1-score	ROC-AUC
Naive baseline	Always Normal	0.804	0.000	0.000	0.000	0.500
Logistic Regression	No SMOTE	0.822	0.600	0.286	0.387	0.709
Logistic Regression	SMOTE	0.636	0.312	0.714	0.435	0.716
Random Forest	No SMOTE	0.813	0.522	0.571	0.545	0.801
Random Forest	**SMOTE**	**0.794**	**0.483**	**0.667**	**0.560**	**0.831**
AdaBoost	No SMOTE	0.785	0.450	0.429	0.439	0.759
AdaBoost	SMOTE	0.748	0.421	0.762	0.542	0.811

The naive baseline achieves high accuracy but completely fails to detect attacks. Logistic Regression with SMOTE improves recall, but its precision and accuracy decrease

significantly. AdaBoost with SMOTE reaches the highest recall, but Random Forest with SMOTE provides the best overall balance, with the highest F1-score and ROC-AUC.

From a cybersecurity perspective, this trade-off is appropriate. A high recall reduces missed attacks, while a moderate false positive rate is acceptable because false alarms can be investigated. Therefore, Random Forest with SMOTE was selected as the final deployed model.

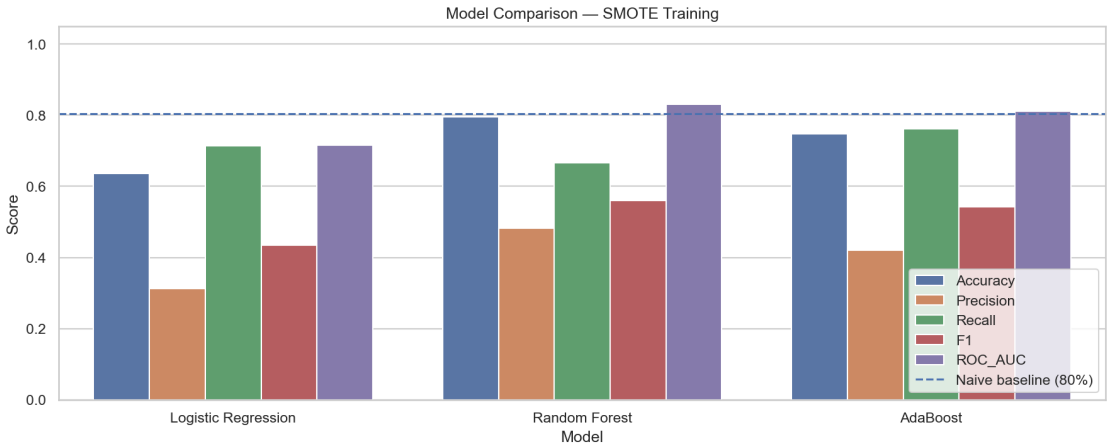

Figure 7: Comparison of evaluation metrics across models. Random Forest with SMOTE provides the strongest balance between F1-score and ROC-AUC.

9 Confusion Matrix and ROC-AUC Analysis

The confusion matrices in Figure 8 show the trade-off between false positives and false negatives. In cybersecurity, false negatives are especially important because they correspond to missed attacks.

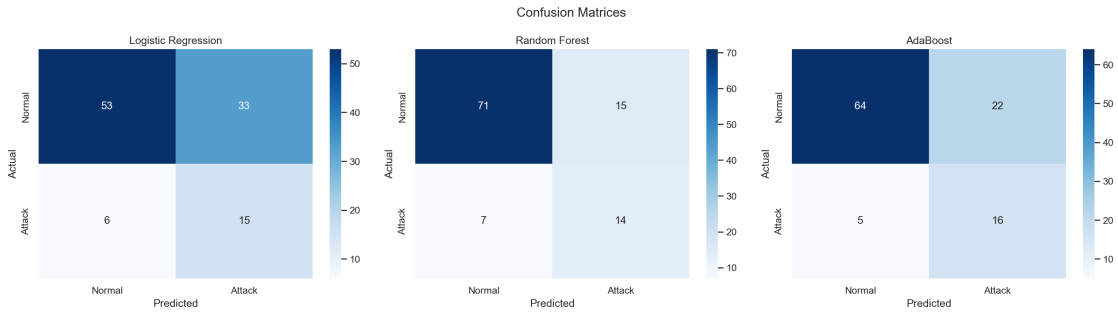

Figure 8: Confusion matrices for the trained models. Random Forest with SMOTE achieves a balanced compromise between detecting attacks and limiting false alarms.

The ROC-AUC curves in Figure 9 show that Random Forest has the strongest separability between normal and attack events, with ROC-AUC equal to 0.831.

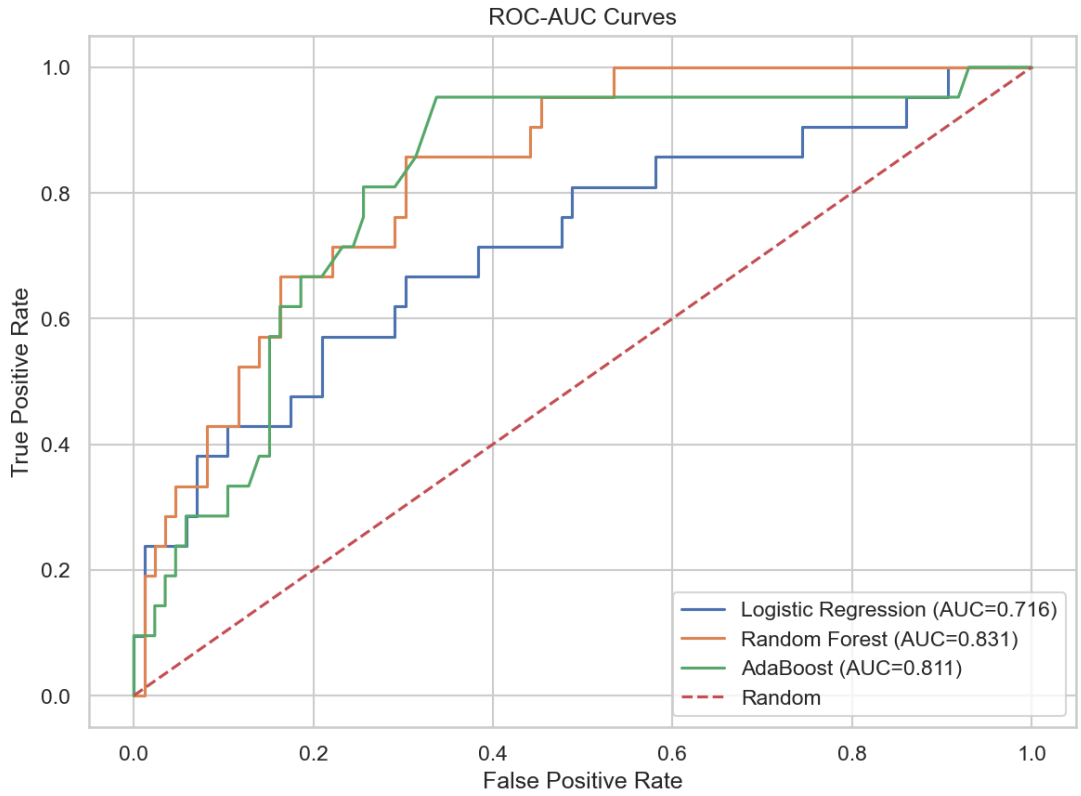

Figure 9: ROC-AUC curves for model comparison. Random Forest with SMOTE achieves the best ROC-AUC, indicating strong class separation.

10 Hyperparameter Tuning and Cross-Validation

To strengthen the model selection process, an additional hyperparameter tuning experiment was performed using `GridSearchCV` and `StratifiedKFold` cross-validation. The tuning focused on Random Forest, testing parameters such as `n_estimators`, `max_depth`, `min_samples_leaf`, and `max_features`.

A key methodological choice was to place SMOTE inside the cross-validation pipeline using `imblearn.pipeline.Pipeline`. This prevents leakage because synthetic samples are generated only inside the training folds, not before validation.

The tuned Random Forest reached a mean cross-validation ROC-AUC of approximately 0.738. However, the deployed Random Forest with SMOTE from the executed notebook remained the best final choice because it achieved stronger holdout performance, especially F1-score = 0.560 and ROC-AUC = 0.831. This shows that the final model was selected based on objective performance rather than complexity alone.

11 Interpretability with SHAP

SHAP was used to explain the Random Forest model. Interpretability is important in cybersecurity because users need to understand why a system classifies an event as an attack.

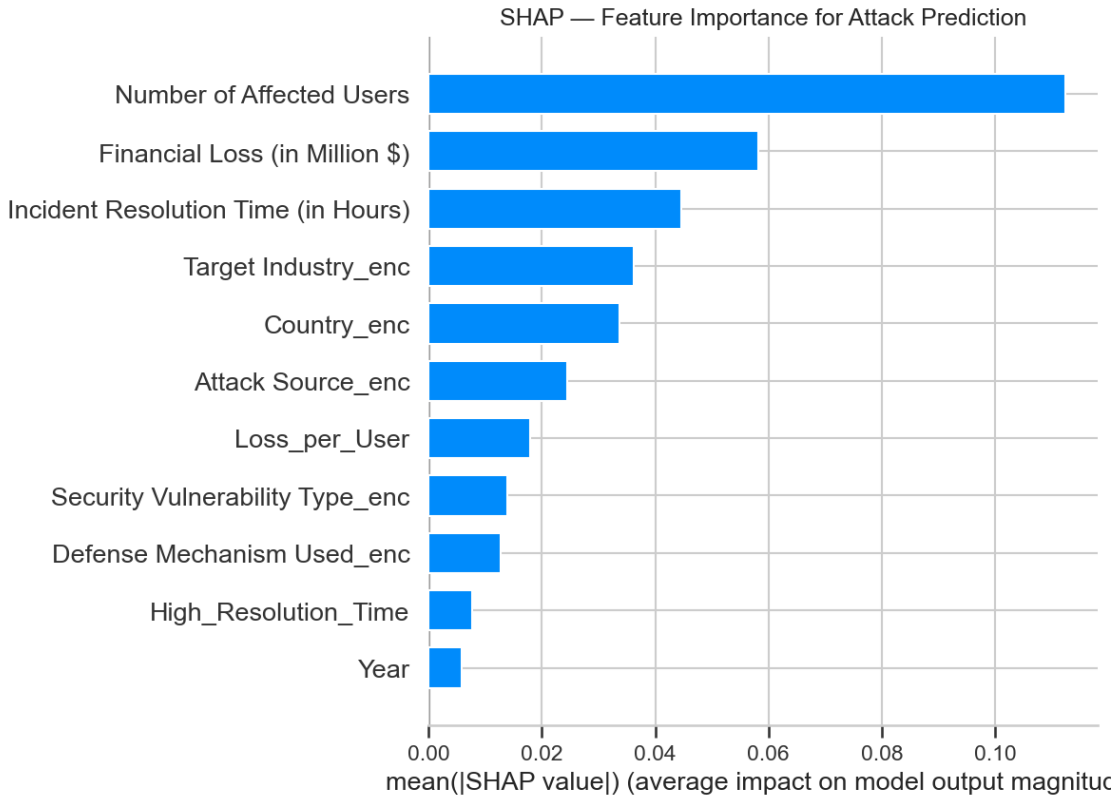

Figure 10: SHAP feature importance plot for the selected Random Forest model. The most influential features are related to event scale, financial impact, resolution time, and encoded categorical context.

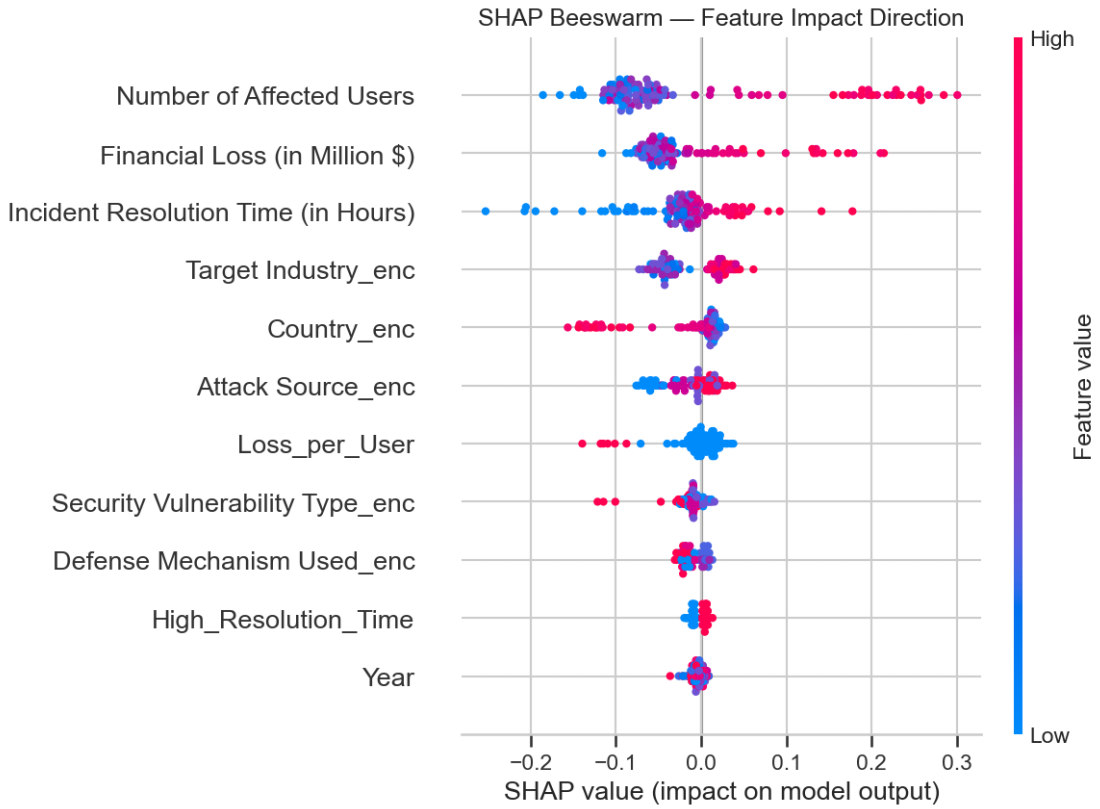

Figure 11: SHAP beeswarm plot showing both feature importance and direction of impact on predictions. Higher risk-related feature values tend to push predictions toward the attack class.

The SHAP analysis shows that the model relies on meaningful cybersecurity patterns. A high number of affected users suggests a larger-scale incident. Higher financial loss indicates stronger incident severity. Longer resolution time suggests more complex attacks requiring more response effort. These interpretations support the reliability and transparency of the selected model.

12 Deployment with FastAPI and Docker

The final model was deployed using a FastAPI REST API and Docker. The backend loads the saved Random Forest model, Logistic Regression model, AdaBoost model, scaler, label encoders, feature columns, median resolution value, and SHAP explainer.

12.1 Dockerfile

The following `Dockerfile` packages all dependencies and exposes the API on port 8000:

```
FROM python:3.10-slim

WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
```

```
COPY . .

EXPOSE 8000

CMD ["uvicorn", "backend:app", "--host", "0.0.0.0", "--port",
   "8000"]
```

12.2 Building and Running the Container

To build and run the Docker container locally, execute the following commands:

```
# Build the image
docker build -t cybersecurity-detector .

# Run the container
docker run -p 8000:8000 cybersecurity-detector

# Test the health check endpoint
curl http://localhost:8000/

# Test the prediction endpoint
curl -X POST http://localhost:8000/predict \
  -H "Content-Type: application/json" \
  -d '{"Country": "USA", "Financial Loss": 50000,
       "Number of Affected Users": 1200,
       "Incident Resolution Time": 48,
       "Attack Source": "External",
       "Target Industry": "Finance"}'
```

The container is self-contained: all model artifacts, encoders, scaler, and the SHAP explainer are copied inside the image at build time. No external dependencies are required at runtime.

Table 6: Main API endpoints implemented in the backend.

Endpoint	Method	Purpose
`/`	GET	Health check
`/options`	GET	Returns valid categorical options
`/predict`	POST	Returns model predictions and explanations

The API returns predictions from the three models, the final Random Forest verdict, attack and normal probabilities, a confidence score, and the top SHAP explanatory features.

12.3 Example API Request

```
POST /predict
{
  "Country": "USA",
  "Financial Loss": 50000,
  "Number of Affected Users": 1200,
  "Incident Resolution Time": 48,
  "Attack Source": "External",
  "Target Industry": "Finance"
}
```

12.4 Example API Response

```
{
  "prediction": "Attack",
  "probability_attack": 0.78,
  "probability_normal": 0.22,
  "confidence": "High",
  "top_features": [
    "Number of Affected Users",
    "Financial Loss",
    "Incident Resolution Time"
  ]
}
```

This deployment demonstrates how the model can be used in a real-time cybersecurity monitoring system.

13 MLflow Tracking

MLflow was used to track all experiments and compare model versions under a single experiment named `cybersecurity_detection`. For each model run, the project logs the model name, training configuration (with/without SMOTE), and all evaluation metrics. Table 7 reproduces the key logged parameters and metrics across runs.

Table 7: MLflow experiment runs — logged parameters and metrics.

Run Name	SMOTE	Accuracy	Precision	Recall	F1-score	ROC-AUC
logistic_regression_no_smote	No	0.822	0.600	0.286	0.387	0.709
logistic_regression_smote	Yes	0.636	0.312	0.714	0.435	0.716
random_forest_no_smote	No	0.813	0.522	0.571	0.545	0.801
random_forest_smote	**Yes**	**0.794**	**0.483**	**0.667**	**0.560**	**0.831**
adaboost_no_smote	No	0.785	0.450	0.429	0.439	0.759
adaboost_smote	Yes	0.748	0.421	0.762	0.542	0.811
rf_tuned_crossval	Yes	–	–	–	–	0.738 (CV mean)

Each run was logged using the following MLflow pattern:

```
with mlflow.start_run(run_name="random_forest_smote"):
    mlflow.log_param("model", "RandomForest")
    mlflow.log_param("smote", True)
    mlflow.log_param("n_estimators", 100)
    mlflow.log_metric("accuracy",  0.794)
    mlflow.log_metric("precision", 0.483)
    mlflow.log_metric("recall",    0.667)
    mlflow.log_metric("f1_score",  0.560)
    mlflow.log_metric("roc_auc",   0.831)
    mlflow.sklearn.log_model(rf_model, "model")
```

The MLflow UI allows visual comparison of all runs, making it straightforward to identify the best-performing configuration. The logged model artifact for `random_forest_smote` is also registered in the MLflow Model Registry for versioned deployment. This tracking setup ensures full reproducibility: any run can be re-examined or re-deployed from its logged parameters and saved artifact.

14 Notebook and Backend Consistency

The notebook and backend are consistent in the main deployment elements. The backend loads the saved `StandardScaler` rather than recreating a new scaler manually. It also loads the saved feature columns and reindexes prediction inputs using the same training feature order.

The deployed feature list is:

```
Year
Financial Loss (in Million $)
Number of Affected Users
Incident Resolution Time (in Hours)
Loss_per_User
High_Resolution_Time
Country_enc
Target Industry_enc
Attack Source_enc
Security Vulnerability Type_enc
Defense Mechanism Used_enc
```

This is important because machine learning models are sensitive to feature order. The backend computes the engineered features `Loss_per_User` and `High_Resolution_Time`, then applies the saved scaler and models.

15 Critical Analysis and Limitations

Although the model achieves strong performance, the results should be interpreted with caution. The main limitation is that normal events are synthetic. They were added because the original dataset contains attack records only. This makes the classification problem more realistic than an attack-only dataset, but it may still introduce bias.

The dataset is also relatively small, and real cybersecurity environments contain more noise, missing values, temporal dependencies, and evolving attack patterns. Therefore, the system is suitable as a complete academic machine learning pipeline, but production use would require validation on real benign traffic and real attack logs.

16 Conclusion

This project successfully implements a complete machine learning pipeline for cybersecurity event detection. It includes realistic binary classification, EDA with visualizations, feature engineering, imbalance handling with SMOTE, baseline comparison, model training, evaluation, SHAP interpretability, MLflow tracking, and FastAPI/Docker deployment.

The final selected model is Random Forest with SMOTE. It achieved **ROC-AUC = 0.831** and **F1-score = 0.560**, making it the best overall compromise between detecting attacks and limiting false alarms. The project also demonstrates methodological care by preventing target leakage, applying SMOTE only on the training set, saving preprocessing artifacts, and keeping deployment transformations consistent with the notebook.

Future improvements could include validation on real-world normal traffic, integration of temporal features, larger-scale datasets, and advanced models such as deep learning or anomaly detection methods.